

DESENVOLVIMENTO WEB / DO ZERO À PRÁTICA

Seu primeiro passo na web.



01

HTML

02

CSS

03

JavaScript

Estrutura. Visual. Interação.

UM GUIA PATODEVS

Conceitos claros, exemplos de código
e um projeto para aprender fazendo.

WEB

Seu caminho de aprendizado

Este guia apresenta as bases para construir páginas web. Você vai escrever arquivos, abrir o resultado no navegador e entender como estrutura, aparência e comportamento trabalham juntos. Não é necessário saber programar antes de começar.

Como estudar: leia um tópico, digite o exemplo e altere uma parte. Observe o resultado antes de avançar. Copiar sem testar não revela se você entendeu o código. Os números à esquerda dos blocos são guias de leitura: não fazem parte do código.

Sumário

- 03 — Como a web funciona
- 04 — Prepare seus arquivos
- 05-07 — HTML: elementos, estrutura e formulários
- 08-10 — CSS: regras, caixas e responsividade
- 11-14 — JavaScript: dados, lógica, coleções e DOM
- 15 — Encontrar e corrigir erros
- 16-19 — Projeto: lista de tarefas
- 20-21 — Exercícios e respostas
- 22 — Glossário e referências
- 23 — Encerramento

Autoria editorial: PatoDevs. Edição introdutória, 2026. Exemplos didáticos: adapte-os antes de utilizá-los em projetos maiores.

Três tecnologias, uma página

Pense em uma lista de tarefas. Ela precisa de um título, um campo e uma lista. Também precisa de espaçamento e cores. Por fim, precisa responder quando alguém adiciona uma tarefa.

HTML: conteúdo e significado

HTML é uma linguagem de marcação. Você descreve as partes do documento: este trecho é um título, aquele é um parágrafo e este outro é um botão. As tags ajudam o navegador a interpretar o conteúdo.

CSS: aparência e organização

CSS define a apresentação: tamanho das letras, cores, bordas, espaço e posição dos elementos. A mesma estrutura HTML pode receber visuais diferentes.

JavaScript: lógica e comportamento

JavaScript é uma linguagem de programação. Ela permite executar cálculos, tomar decisões e responder a ações como digitar, clicar ou enviar um formulário.

O navegador faz a ligação

Ao abrir a página, o navegador lê o HTML e carrega os arquivos associados. Ele constrói uma representação do documento, aplica os estilos e executa o JavaScript. Você verá essa representação, chamada DOM, na página 14.

PRATIQUE

Em uma tela de login, identifique: o campo pertence à estrutura; a borda pertence ao visual; a reação ao enviar pertence ao comportamento. Pense em outro exemplo com as três partes.

Prepare seus arquivos

Você precisa de um editor de texto ou código e de um navegador. Crie uma pasta chamada **primeiro-site**. Dentro dela, crie três arquivos. Use exatamente os mesmos nomes nos links do HTML.

```
01 primeiro-site/  
02   index.html  
03   style.css  
04   app.js
```

index.html contém o documento. **style.css** guarda os estilos. **app.js** reúne o comportamento. Essa separação facilita encontrar o que você quer modificar.

Seu ciclo de trabalho

1. Escreva e salve o arquivo.
2. Abra index.html no navegador.
3. Faça uma alteração no editor e salve.
4. Atualize a página para ver o resultado.

Em editores comuns no Windows, confirme que o arquivo não foi salvo como index.html.txt. Um caminho relativo, como style.css, procura o arquivo a partir da localização do HTML. Nomes e letras maiúsculas devem corresponder aos arquivos.

Ferramentas do navegador

Abra as ferramentas de desenvolvimento pelo menu do navegador. A aba de elementos permite inspecionar a estrutura e os estilos. O console mostra mensagens e erros de JavaScript. Alterações feitas apenas no inspetor normalmente desaparecem ao recarregar: salve a mudança no código.

PRATIQUE

Crie os três arquivos. Escreva “Olá, PatoDevs!” no HTML, abra-o e confirme que está editando o arquivo correto.

Entenda os elementos

Um elemento costuma ter uma tag de abertura, conteúdo e uma tag de fechamento. A barra no fechamento indica o fim daquele elemento.

```
01 | <h1>Minha primeira página</h1>  
02 | <p>Estou aprendendo desenvolvimento web.</p>
```

h1 representa um título de nível 1. **p** representa um parágrafo. Escolha a tag pelo significado. O tamanho que o navegador aplica por padrão pode ser alterado com CSS.

Atributos acrescentam informações

```
01 | <a href="https://developer.mozilla.org/">  
02 |   Consultar documentação  
03 | </a>  
04 | 
```

href informa o destino do link. **src** informa o caminho da imagem. **alt** descreve uma imagem relevante; imagens puramente decorativas podem usar alt vazio. A tag **img** é um elemento vazio: não recebe uma tag de fechamento.

Uma coisa dentro da outra

```
01 | <p>Aprender exige <strong>prática</strong>.</p>
```

O elemento **strong** marca uma parte importante do texto. Feche os elementos na ordem correta: o que foi aberto por último, quando aninhado, é fechado primeiro.

PRATIQUE

Crie um título, dois parágrafos e um link. Altere o destino do link. Se uma imagem não aparecer, confira o caminho e o nome do arquivo.

Monte um documento completo

```
01 <!doctype html>
02 <html lang="pt-BR">
03 <head>
04   <meta charset="utf-8">
05   <meta name="viewport"
06     content="width=device-width, initial-scale=1">
07   <title>Minha página</title>
08   <link rel="stylesheet" href="style.css">
09   <script src="app.js" defer></script>
10 </head>
11 <body>
12   <header><h1>Meu site</h1></header>
13   <main>
14     <h2>Sobre mim</h2>
15     <p>Meu primeiro projeto na web.</p>
16   </main>
17   <footer>Feito por mim.</footer>
18 </body>
19 </html>
```

doctype solicita o modo de renderização padrão. **lang** indica o idioma. **charset** define a codificação. **viewport** ajuda a página a se adaptar à largura do dispositivo.

head guarda metadados e referências; **body** contém o conteúdo da página. **title** nomeia a aba. O atributo **defer** faz este script externo executar após o HTML ser analisado.

header, main e footer descrevem o papel das regiões. Uma div é um agrupador genérico: use quando não houver um elemento com significado mais adequado. Mantenha uma hierarquia coerente de títulos.

PRATIQUE

Copie a estrutura, altere o título da aba e o texto da página. Confirme que title e h1 têm funções diferentes.

Receba informações com formulários

```
01 <form id="cadastro">
02   <label for="nome">Seu nome</label>
03   <input id="nome" name="nome" required>
04   <button type="submit">Enviar</button>
05 </form>
```

O **form** agrupa controles. O **label** identifica o campo e usa **for** para apontar para o **id** do **input**. Clique no rótulo: o campo associado recebe foco. Não dependa apenas do placeholder, que desaparece ao digitar.

id identifica um elemento e deve ser único na página. **name** identifica o dado na submissão do formulário. **required** indica que o campo deve ser preenchido.

Tipos de campo e ações

```
01 <input type="email" name="email" required>
02 <input type="number" name="quantidade" min="1">
03 <button type="button">Abrir ajuda</button>
```

type="email" ajuda a validar o formato do endereço. **type="number"** fornece um controle numérico. Um botão **submit** envia o formulário; um botão **button** serve para outra ação. Declare o tipo para evitar envios inesperados.

A validação do navegador melhora a experiência, mas pode ser contornada. Em um sistema com servidor, os dados também precisam ser validados no backend. No projeto deste livro, a submissão será tratada pelo JavaScript.

PRATIQUE

Adicione um campo de e-mail com rótulo. Tente enviar vazio e com um formato inválido. Navegue pelos campos usando Tab.

Selecione e estilize

Uma regra CSS contém um seletor e um bloco de declarações. Cada declaração associa uma propriedade a um valor. O ponto e vírgula separa as declarações.

```
01 p {  
02   color: #333333;  
03   font-size: 16px;  
04   line-height: 1.6;  
05 }  
06 .destaque { background: #eeeeee; }  
07 #titulo { margin-bottom: 24px; }
```

p seleciona elementos desse tipo. **.destaque** seleciona elementos com `class="destaque"`. **#titulo** seleciona o elemento com `id="titulo"`. Classes podem ser reutilizadas em vários elementos.

Quando duas regras disputam

Em regras comuns de autor, sem camadas ou `!important`, a especificidade ajuda a decidir qual declaração vence. Um seletor de id tem maior especificidade que uma classe; uma classe tem maior especificidade que um seletor de tipo. Em empate, a declaração posterior vence.

```
01 .aviso { color: gray; }  
02 .aviso { color: black; } /* vence neste empate */
```

Algumas propriedades, como `color`, podem ser herdadas do elemento pai. Outras, como `margin`, não são herdadas normalmente. Não use `!important` para corrigir tudo: primeiro descubra qual regra está sendo aplicada no inspetor.

PRATIQUE

Aplique uma classe a dois parágrafos. Mude a cor dessa classe e observe os dois. Depois escreva uma segunda regra igual com outra cor.

Entenda as caixas e os espaços

No layout CSS, pense em cada elemento como uma caixa. O modelo de caixa inclui conteúdo, padding, border e margin. Entender essas partes evita ajustar o layout por tentativa e erro.

```
01 * { box-sizing: border-box; }
02 .cartao {
03     width: 300px;
04     padding: 20px;
05     border: 1px solid #cccccc;
06     margin: 24px 0;
07 }
```

Conteúdo: texto, imagem ou outros elementos.

Padding: espaço entre conteúdo e borda.

Border: contorno da caixa.

Margin: espaço externo, entre esta caixa e outras.

Com border-box, a largura declarada inclui padding e borda. No exemplo, a caixa mede 300px de largura até a borda; a margem fica fora. Sem border-box, a largura de 300px normalmente se refere ao conteúdo, e padding e borda aumentam a medida externa.

Um conteúdo centralizado

```
01 main {
02     max-width: 720px;
03     margin: 0 auto;
04     padding: 24px;
05 }
```

max-width limita a largura. Margens horizontais automáticas centralizam uma caixa de bloco quando há espaço disponível. O padding mantém o texto afastado das bordas. Evite definir uma largura fixa maior que a tela.

PRATIQUE

Troque o padding do cartão de 20px para 40px. Com border-box, observe o espaço do conteúdo mudar sem a largura externa crescer.

Organize e adapte o layout

Flexbox organiza itens principalmente em uma direção: linha ou coluna. É útil para menus, grupos de botões e pequenas composições. As propriedades do exemplo atuam no contêiner e organizam seus filhos diretos.

```
01 .acoes {  
02   display: flex;  
03   gap: 12px;  
04   align-items: center;  
05   justify-content: space-between;  
06   flex-wrap: wrap;  
07 }
```

gap cria espaço entre itens. **align-items** alinha no eixo transversal. **justify-content** distribui no eixo principal. **flex-wrap** permite quebrar os itens em mais de uma linha.

Um layout que responde à tela

```
01 .acoes { flex-direction: row; }  
02 @media (max-width: 600px) {  
03   .acoes { flex-direction: column; }  
04 }  
05 img { max-width: 100%; height: auto; }
```

A media query aplica regras quando a condição é atendida. Neste caso, as ações ficam em coluna em telas de até 600px. A imagem não ultrapassa a largura disponível e mantém a proporção. Responsividade envolve conteúdo, leitura e controles, além de mudar tamanhos.

Teste larguras pequenas e grandes. Use texto de verdade, títulos longos e navegação por teclado. Preserve um foco visível e contraste suficiente; não comunique informações apenas pela cor.

PRATIQUE

Monte um grupo com três botões. Reduza a largura do navegador e confirme que eles continuam acessíveis, sem rolagem horizontal.

Guarde e transforme dados

Uma variável associa um nome a um valor. Use `const` quando você não pretende reatribuir esse nome. Use `let` quando o valor associado precisa mudar. Os nomes devem explicar o propósito do dado.

```
01 | const nome = "Ana";           // string: texto
02 | let pontos = 10;              // number: número
03 | const ativo = true;           // boolean: verdadeiro/falso
04 | pontos = pontos + 5;
05 | console.log(nome, pontos); // Ana, 15
```

Strings usam aspas. Numbers permitem cálculos. Booleans representam `true` ou `false`. O `console.log` exibe informações no console: use para investigar um valor, e não como interface para o usuário.

Texto e número não são iguais

```
01 | console.log("2" + 3); // "23": concatenação
02 | console.log(2 + 3);   // 5: soma
03 | const quantidade = Number("2");
04 | console.log(quantidade * 3); // 6
```

Valores de campos de texto chegam como strings. Converta quando precisar calcular e valide o resultado: `Number("abc")` produz `NaN`, que representa um resultado numérico inválido. Um nome declarado com `const` não pode ser reatribuído; isso não torna imutáveis todos os dados de um objeto.

PRATIQUE

Crie `preco = 20` e `quantidade = 3`. Calcule o total. Depois troque um número por texto e veja por que a conversão importa.

Tome decisões e crie funções

Uma condição escolhe qual trecho executar. O operador `===` compara sem converter automaticamente os tipos. Operadores como `>`, `<` e `>=` ajudam a comparar números.

```
01 | const nota = 8;
02 | if (nota >= 7) {
03 |     console.log("Aprovado");
04 | } else {
05 |     console.log("Precisa revisar");
06 | }
```

A expressão entre parênteses é avaliada. Se for verdadeira, o primeiro bloco executa; caso contrário, o bloco `else` executa. Use `&&` para combinar condições que precisam ser verdadeiras e `||` quando basta uma delas.

Funções dão nome a uma operação

```
01 | function calcularTotal(preco, quantidade) {
02 |     return preco * quantidade;
03 | }
04 | const total = calcularTotal(20, 3);
05 | console.log(total); // 60
```

preco e **quantidade** são parâmetros: nomes para os valores recebidos. Os valores 20 e 3 são argumentos da chamada. **return** devolve o resultado e encerra a execução da função. Declarar uma função não executa seu corpo: é preciso chamá-la.

Uma função pequena, com propósito claro, é mais fácil de testar e reutilizar. Antes de escrever, explique a operação em uma frase: “receber preço e quantidade e devolver o total”.

PRATIQUE

Crie uma função que receba uma idade e devolva `true` quando ela for maior ou igual a 18. Teste com 17, 18 e 19.

Trabalhe com coleções

Arrays guardam sequências de valores. O primeiro item fica no índice zero. Objetos agrupam propriedades relacionadas. Você pode combinar os dois para representar vários registros.

```
01 const tarefas = ["Estudar", "Praticar"];
02 console.log(tarefas[0]); // Estudar
03 console.log(tarefas.length); // 2
04 const pessoa = { nome: "Ana", idade: 20 };
05 console.log(pessoa.nome); // Ana
```

Repita uma ação para cada item

```
01 for (const tarefa of tarefas) {
02   console.log(tarefa);
03 }
04 tarefas.push("Revisar");
05 console.log(tarefas.length); // 3
```

O for...of percorre os valores da coleção. push adiciona um item ao final do array. Um array declarado com const pode ter seu conteúdo alterado; o que não pode mudar é a associação do nome para outro array.

Encontre os dados que interessam

```
01 const pedidos = [
02   { cliente: "Ana", pago: true },
03   { cliente: "Bia", pago: false }
04 ];
05 const pendentes = pedidos.filter(pedido => !pedido.pago);
06 console.log(pendentes.length); // 1
```

filter cria um novo array com os itens que passam na condição. A função com => é uma arrow function. Nesse exemplo, ela recebe cada pedido e devolve se ele está pendente. O operador ! inverte o valor lógico.

PRATIQUE

Adicione um terceiro pedido não pago. Confira o tamanho de pendentes. Explique por que o array pedidos continua com todos os registros.

Conecte o código à página

DOM é a representação do documento que o navegador disponibiliza ao JavaScript. Você pode selecionar elementos, ler seus valores e alterar seu conteúdo. Os métodos abaixo fazem parte das APIs do navegador.

```
01 <!-- HTML dentro do body -->
02 <button id="saudar" type="button">Saudar</button>
03 <p id="mensagem" role="status"></p>
```

```
01 // JavaScript em app.js, carregado com defer
02 const botao = document.querySelector("#saudar");
03 const mensagem = document.querySelector("#mensagem");
04 botao.addEventListener("click", () => {
05     mensagem.textContent = "Olá, PatoDevs!";
06 });
```

querySelector encontra o primeiro elemento correspondente ao seletor. **addEventListener** registra uma função para responder a um evento. A função passada não é executada imediatamente: ela roda quando o clique acontece.

textContent define texto, sem interpretá-lo como HTML. Para mostrar conteúdo digitado pelo usuário, prefira essa abordagem a inserir strings em `innerHTML`. Se o seletor não encontrar o elemento, ele retorna `null`; chamar um método sobre `null` gera erro.

Estilo continua no CSS

```
01 mensagem.classList.add("destaque");
02 mensagem.classList.remove("destaque");
```

`classList` permite mudar classes. Assim, o JavaScript decide o estado e o CSS define sua aparência. O atributo `role="status"` permite anunciar atualizações desse texto a tecnologias assistivas.

PRATIQUE

Troque o texto exibido. Depois adicione a classe destaque ao clicar e crie a regra correspondente no CSS.

Encontre o erro com método

Um erro não significa que você não consegue programar. Significa que o comportamento observado não corresponde ao que você esperava. Reduza o problema e investigue uma hipótese por vez.

CSS não aparece

Confira se o link aponta para style.css, se o arquivo foi salvo e se o seletor corresponde ao HTML. No inspetor, procure a declaração: ela pode estar ausente, inválida ou sobrescrita. Exemplo de erro de seletor: escrever .titulo quando o HTML possui id="titulo".

JavaScript não funciona

Abra o console e leia a primeira mensagem de erro. Verifique o nome do arquivo e a linha informada. Confirme se o seletor encontra o elemento e se o script usa defer. Um erro inicial pode impedir as instruções seguintes de executar.

```
01 | const campo = document.querySelector("#nome");  
02 | console.log(campo); // elemento ou null  
03 | console.log(campo.value); // texto do campo, se encontrado
```

O resultado está errado

Exiba os valores de entrada e os tipos com typeof. Confira se você está somando números ou juntando textos. Para uma função de total, teste uma entrada pequena cujo resultado você consegue calcular à mão.

Checklist antes de entregar: arquivos carregam; campos possuem rótulos; navegação por Tab funciona; a página cabe no celular; ações têm retorno; entradas vazias são tratadas; o console não apresenta erros inesperados.

PRATIQUE

Mude intencionalmente um id usado pelo JavaScript. Observe o erro, identifique a causa e corrija. Explique a diferença entre sintaxe inválida e lógica incorreta.

Construa uma lista de tarefas

Vamos juntar as três tecnologias em uma aplicação pequena. Você poderá adicionar uma tarefa e remover um item concluído. A lista existirá apenas enquanto a página estiver aberta: ao recarregar, ela será reiniciada.

O que você vai praticar

HTML: formulário, rótulo, botão e lista.

CSS: largura fluida, espaçamento, Flexbox e foco.

JavaScript: evento submit, leitura de texto, condição, criação e remoção de elementos.

Monte a pasta

```
01 | tarefas/  
02 |   index.html  
03 |   style.css  
04 |   app.js
```

As próximas três páginas trazem os arquivos completos. Copie cada bloco para o arquivo indicado. Mantenha os nomes e os ids iguais aos do exemplo. Depois abra index.html e adicione sua primeira tarefa.

Critérios de funcionamento

1. Adicionar “Estudar HTML” cria um item na lista.
2. Tentar adicionar somente espaços mostra uma orientação.
3. A ação Concluir remove apenas o item selecionado.
4. Após adicionar, o campo fica vazio e recebe foco.
5. Em uma tela estreita, os controles continuam utilizáveis.

PRATIQUE

Antes de copiar o JavaScript, descreva a lógica: receber o envio, ler o campo, rejeitar texto vazio, criar o item, adicionar à lista e limpar o campo.

index.html — a estrutura

```
01 <!doctype html>
02 <html lang="pt-BR">
03 <head>
04   <meta charset="utf-8">
05   <meta name="viewport"
06     content="width=device-width, initial-scale=1">
07   <title>Minhas tarefas | PatoDevs</title>
08   <link rel="stylesheet" href="style.css">
09   <script src="app.js" defer></script>
10 </head>
11 <body>
12   <main>
13     <h1>Minhas tarefas</h1>
14     <form id="formulario">
15       <label for="tarefa">Nova tarefa</label>
16       <div class="entrada">
17         <input id="tarefa" maxlength="120"
18           placeholder="O que você precisa fazer?">
19         <button type="submit">Adicionar</button>
20       </div>
21     </form>
22     <p id="aviso" role="status"></p>
23     <ul id="lista" aria-label="Tarefas"></ul>
24     <footer>Projeto de estudo – PatoDevs</footer>
25   </main>
26 </body>
27 </html>
```

O formulário permite adicionar pelo botão ou pela tecla Enter. O rótulo identifica o campo. O aviso será atualizado pelo JavaScript, e a lista receberá os itens criados. maxlength limita a entrada a 120 caracteres.

Neste exemplo, a entrada vazia será tratada pelo JavaScript para demonstrar a validação manual. O script é carregado com defer, então os elementos já estarão disponíveis ao executar.

style.css — o visual

```
01  * { box-sizing: border-box; }
02  body {
03    margin: 0;
04    font-family: Arial, sans-serif;
05    background: #f5f5f5;
06    color: #222;
07  }
08  main {
09    max-width: 560px;
10    margin: 48px auto;
11    padding: 24px;
12  }
13  label { display: block; margin-bottom: 8px; }
14  .entrada { display: flex; gap: 8px; }
15  input { flex: 1; min-width: 0; }
16  input, button { padding: 12px; font: inherit; }
17  button { cursor: pointer; }
18  ul { list-style: none; padding: 0; }
19  li {
20    display: flex;
21    align-items: center;
22    gap: 12px;
23    padding: 12px 0;
24    border-bottom: 1px solid #ccc;
25  }
26  li span { flex: 1; overflow-wrap: anywhere; }
27  :focus-visible { outline: 2px solid #222; }
28  footer { margin-top: 32px; font-size: 12px; }
29  @media (max-width: 420px) {
30    .entrada { flex-direction: column; }
31  }
```

`min-width: 0` permite que o campo encolha no Flexbox. `overflow-wrap` evita que textos longos ultrapassem o item. Em telas estreitas, o campo e o botão ficam um abaixo do outro. O foco visível ajuda a identificar o controle selecionado pelo teclado.

app.js — o comportamento

```
01 const formulario = document.querySelector("#formulario");
02 const campo = document.querySelector("#tarefa");
03 const lista = document.querySelector("#lista");
04 const aviso = document.querySelector("#aviso");
05
06 formulario.addEventListener("submit", (evento) => {
07     evento.preventDefault();
08     const texto = campo.value.trim();
09     if (texto === "") {
10         aviso.textContent = "Escreva uma tarefa.";
11         campo.focus();
12         return;
13     }
14     const item = document.createElement("li");
15     const descricao = document.createElement("span");
16     descricao.textContent = texto;
17     const botao = document.createElement("button");
18     botao.type = "button";
19     botao.textContent = "Concluir";
20     botao.setAttribute("aria-label", `Concluir: ${texto}`);
21     botao.addEventListener("click", () => {
22         item.remove();
23         aviso.textContent = "Tarefa concluída.";
24         campo.focus();
25     });
26     item.append(descricao, botao);
27     lista.append(item);
28     campo.value = "";
29     campo.focus();
30     aviso.textContent = "Tarefa adicionada.";
31 });
```

preventDefault impede o envio padrão que recarregaria a página. **trim** remove espaços das extremidades. **return** interrompe a função se não houver texto. **createElement** cria elementos e **append** os insere. Cada botão guarda uma referência ao seu próprio item, por isso remove somente aquela tarefa.

Teste texto vazio, texto com espaços e duas tarefas iguais. Elas são itens independentes. Recarregar apaga a lista: persistência será um assunto para a próxima etapa de estudo.

EXERCÍCIOS

20

Teste o que você entendeu

Tente responder e programar antes de consultar a próxima página. Use o console para conferir os resultados e explique cada resposta com suas palavras.

01 / Estrutura

Crie um título “Meu perfil”, um parágrafo de apresentação e um link. Que atributo indica o destino do link? Por que a aparência não deve decidir qual nível de título usar?

02 / Estilo

Crie uma classe chamada `cartao` com padding de 16px, borda de 1px e margin de 20px. Explique qual espaço é interno e qual é externo. Aplique a classe a um agrupador HTML.

03 / Cálculo

Crie a função `calcularTotal(preco, quantidade)`. Qual deve ser o resultado para 15 e 4? O que muda se você usar `"15" + 4` em vez de multiplicar números?

04 / Decisão

Uma compra recebe frete grátis quando o total é maior ou igual a 100. Escreva uma função que devolva `true` ou `false`. Teste 99, 100 e 101.

05 / Coleções

Crie um array com três nomes. Use `for...of` para mostrar cada nome. Qual é o índice do primeiro item? Qual propriedade informa a quantidade de itens?

06 / Evolua o projeto

Mostre quantas tarefas estão na lista. Atualize o contador ao adicionar e concluir. Dica: `lista.children.length` informa a quantidade de elementos filhos. Faça o contador iniciar em zero.

PRATIQUE

Não avance apenas porque o código funcionou. Você consegue explicar o papel de cada variável e prever o resultado de uma pequena mudança?

RESPOSTAS

21

Confira e compare

01. Use `h1`, `p` e `a`. O destino fica em `href`. Títulos devem representar a hierarquia do conteúdo; o CSS define sua aparência.

```
01 | <h1>Meu perfil</h1>
02 | <p>Estou aprendendo a criar páginas.</p>
03 | <a href="https://developer.mozilla.org/">Estudar</a>
```

02. `padding` é interno; `margin` é externo. A borda separa o conteúdo com `padding` da região externa.

```
01 | .cartao { padding: 16px; border: 1px solid; margin: 20px; }
```

03. A função retorna `preco * quantidade`: 15×4 é 60. A expressão `"15" + 4` resulta em `"154"`, pois o operador `+` concatena com uma string.

04. Retorne `total >= 100`. Os resultados para 99, 100 e 101 são `false`, `true` e `true`.

```
01 | function temFreteGratis(total) {
02 |     return total >= 100;
03 | }
```

05. O primeiro índice é 0 e a propriedade é `length`. Exemplo: `for (const nome of nomes) { console.log(nome); }`.

06. Adicione um parágrafo com `id="contador"` ao HTML. No JavaScript, selecione-o e crie a função abaixo. Chame-a uma vez na inicialização, após `lista.append(item)` e após `item.remove()`.

```
01 | const contador = document.querySelector("#contador");
02 | function atualizarContador() {
03 |     contador.textContent = `Tarefas: ${lista.children.length}`;
04 | }
```

Teste: início 0; duas adições 2; uma conclusão 1. Existem outras soluções corretas. Compare os resultados e a clareza do código.

Glossário e próximos passos

Tag: marca que participa da estrutura de um elemento HTML.

Atributo: informação adicional de um elemento.

Seletor: padrão para localizar elementos no CSS ou no DOM.

Propriedade CSS: característica de apresentação.

Variável: nome associado a um valor.

Função: bloco de código que pode ser chamado.

Evento: sinal de uma ação ou ocorrência no navegador.

DOM: representação programável do documento.

Responsividade: adaptação do layout a diferentes condições de tela.

Continue construindo

Depois deste guia, pratique uma página de apresentação, um formulário com mensagens de validação e uma lista com filtros. Estude CSS Grid, métodos de arrays, acessibilidade e armazenamento local. Mais adiante, explore requisições HTTP, APIs e backend. Aprenda uma necessidade por vez, testando o resultado.

Referências para aprofundar

Os exemplos e a organização didática deste material são próprios. Consulte a documentação técnica para explorar os conceitos em maior detalhe. Referências consultadas em 5 de outubro de 2026.

[MDN Web Docs — HTML: estrutura e semântica](#)

[MDN Web Docs — CSS: fundamentos](#)

[MDN Web Docs — JavaScript: primeiros passos](#)

[MDN Web Docs — DOM: manipulação do documento](#)

PatoDevs

HTML, CSS e JavaScript básico — Seu primeiro passo na web.

Aprenda, experimente e construa seu próximo projeto.

Seu próximo passo é criar.

Você chegou ao fim deste guia e conheceu as três bases de uma página web: o HTML organiza o conteúdo, o CSS define a apresentação e o JavaScript acrescenta lógica e interação.

Se você acompanhou os exemplos e construiu a lista de tarefas, já deu um passo importante: transformar conceitos em algo que funciona. Agora, procure entender o que escreveu e experimentar suas próprias mudanças.

Não precisa saber tudo para começar.

Volte aos pontos que ainda ficaram confusos. Teste uma ideia pequena, observe o resultado e corrija o que não funcionou. Encontrar erros, consultar a documentação e refazer uma parte também fazem parte do aprendizado.

Seu próximo projeto pode ser simples: uma página sobre você, uma calculadora ou uma lista com novas funções. Escolha algo que consiga terminar e explique como cada parte funciona.

O aprendizado continua quando você coloca a mão no código.

Obrigado por aprender com a PatoDevs. Que este guia seja o começo dos projetos que você ainda vai construir.

